

Config

mod config

FUNCTION	DESCRIPTION
<code>get_store</code>	Access LangGraph store from inside a graph node or entrypoint task at runtime.
<code>get_stream_writer</code>	Access LangGraph <code>StreamWriter</code> from inside a graph node or entrypoint task at runtime.

func get_store

```
get_store() -> BaseStore
```



Access LangGraph store from inside a graph node or entrypoint task at runtime.

Can be called from inside any `StateGraph` node or functional API `task`, as long as the `StateGraph` or the `entrypoint` was initialized with a store, e.g.:

```
# with StateGraph
graph = (
    StateGraph(...)
    ...
    .compile(store=store)
)

# or with entrypoint
@entrypoint(store=store)
def workflow(inputs):
    ...
```



⚠ Async with Python < 3.11

If you are using Python < 3.11 and are running LangGraph asynchronously, `get_store()` won't work since it uses `contextvar` propagation (only available in [Python >= 3.11](#)).

🔧 Using with StateGraph

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START
from langgraph.store.memory import InMemoryStore
from langgraph.config import get_store

store = InMemoryStore()
store.put(("values",), "foo", {"bar": 2})

class State(TypedDict):
    foo: int

def my_node(state: State):
    my_store = get_store()
    stored_value = my_store.get(("values",), "foo").value["bar"]
    return {"foo": stored_value + 1}

graph = (
    StateGraph(State)
    .add_node(my_node)
    .add_edge(START, "my_node")
    .compile(store=store)
)

graph.invoke({"foo": 1})
```

```
{"foo": 3}
```

**Using with functional API**

```

from langgraph.func import entrypoint, task
from langgraph.store.memory import InMemoryStore
from langgraph.config import get_store

store = InMemoryStore()
store.put(("values",), "foo", {"bar": 2})

@task
def my_task(value: int):
    my_store = get_store()
    stored_value = my_store.get(("values",), "foo").value["bar"]
    return stored_value + 1

@entrypoint(store=store)
def workflow(value: int):
    return my_task(value).result()

workflow.invoke(1)

```

3

func get_stream_writer

```
get_stream_writer() -> StreamWriter
```

Access LangGraph `StreamWriter` from inside a graph node or entrypoint task at runtime.

Can be called from inside any `StateGraph` node or functional API `task`.

**Async with Python < 3.11**

If you are using Python < 3.11 and are running LangGraph asynchronously, `get_stream_writer()` won't work since it uses `contextvar` propagation (only available in Python >= 3.11).

**Using with** StateGraph

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START
from langgraph.config import get_stream_writer

class State(TypedDict):
    foo: int

def my_node(state: State):
    my_stream_writer = get_stream_writer()
    my_stream_writer({"custom_data": "Hello!"})
    return {"foo": state["foo"] + 1}

graph = (
    StateGraph(State)
    .add_node(my_node)
    .add_edge(START, "my_node")
    .compile(store=store)
)

for chunk in graph.stream({"foo": 1}, stream_mode="custom"):
    print(chunk)
```

```
{"custom_data": "Hello!"}
```



Using with functional API



```
from langgraph.func import entrypoint, task
from langgraph.config import get_stream_writer

@task
def my_task(value: int):
    my_stream_writer = get_stream_writer()
    my_stream_writer({"custom_data": "Hello!"})
    return value + 1

@entrypoint(store=store)
def workflow(value: int):
    return my_task(value).result()

for chunk in workflow.stream(1, stream_mode="custom"):
    print(chunk)
```

```
{"custom_data": "Hello!"}
```